Nightpass

Unlinkable prepaid API credits for AI agents — on Midnight

Pay for every API call with a zero-knowledge proof.
The provider learns "paid by a valid, unspent credit" — never *who*.

Midnight Buildathon · Wave 3 · Track: Build Privacy-First Apps

The problem

- AI agents now pay per API call (x402 micropayments, API keys tied to wallets).
- Every paid call is linkable: which agent, which endpoint, how often, when.
- For an agent, that log *is* its strategy: trading signals, research targets, customers.
- Today the only options are: trust the provider, or don't use paid APIs.

The idea

- **Buy once:** the agent buys a bundle of *N* calls; the agent sends only `ownerTag = H(secret)`; the circuit stores the commitment `H(ownerTag, N)`. Self-serve `buy()` pays NIGHT into the contract.
- **Spend anonymously:** each call = one ZK `redeem()` proving "I own an unspent use of *some* bundle in the tree".
- **No double spend:** a per-use nullifier `H(secret, i)` goes into a public set.
- **Bound to the request:** `sha256(nonce || claim)` is recorded next to the nullifier → only the paying agent can collect the response (no front-running).

Dual-ledger design

	Public ledger (on-chain)	Private state (agent only)
Bundle	commitment in `HistoricMerkleTree<16>`	secret, N, next index
Spend	nullifier in `Set`, nullifier → request tag in `Map`	which bundle, which use, Merkle path
Counters	`bundlesIssued`, `creditsRedeemed`	credits left
Operator	hash of the operator key (sealed)	operator secret

Witnesses: `bundle()`, `bundlePath()`, `useIndex()`, `localSecret()`. Historic roots keep old bundles spendable.

What the circuit enforces

- ✔ path leaf = `bundleCommitment(bundle)` → N cannot be inflated
- ✔ root is a current or past root of the tree → only paid bundles
- ✔ `useIndex < N` → exactly N spends
- ✔ nullifier unspent → no replay / double spend
- ✔ commitment built in-circuit from (ownerTag, N) → no inflation via `issue()` or `buy()`
- ✔ only the operator can `issue()` / `withdraw()`; `buy()` must pay N × price NIGHT

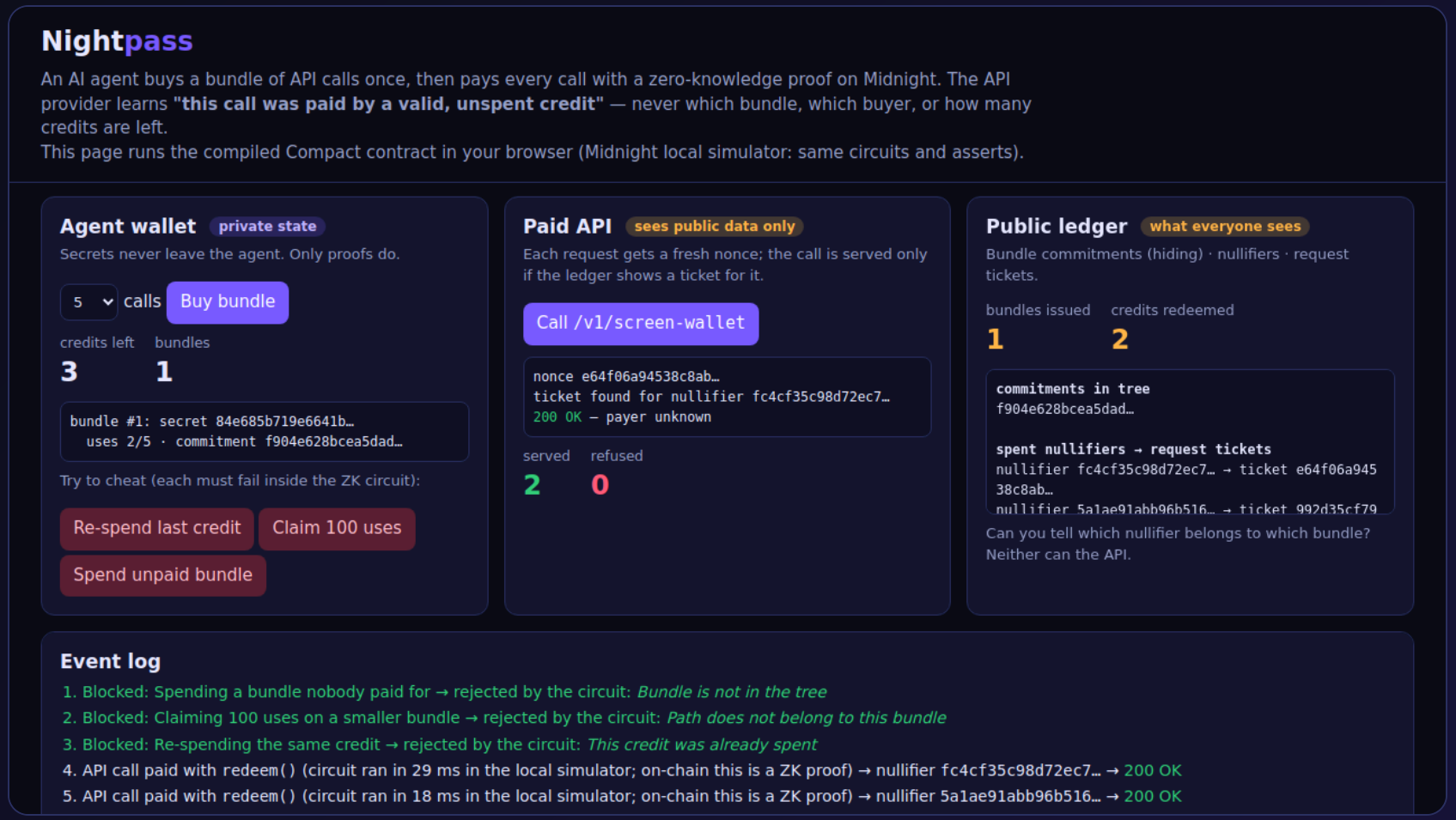
21 tests (16 contract + 5 HTTP paywall) cover every rule plus unlinkability, front-running, request binding and concurrent replays — all passing.

Live on the public Midnight preview network

Step (real ZK proofs, preview network)	Result
deploy, price 1000 / credit	contract 152c8498…56b3
`issue(ownerTag, 3)`	block 1618076
`redeem` credit #0	block 1618081
`redeem` credit #0 again	rejected: already spent
`redeem` credit #1	block 1618085
`buy(ownerTag, 2)` paying 2000 NIGHT units	block 1618089
`withdraw(1500)` (in-circuit balance check)	block 1618093

Also on preview (contract 402c90…) and a local chain: HTTP paywall reading the indexer: 402 → on-chain redeem → 200; ticket replayed without the claim → 402; replay → 403.
Live chain view (decodes the deployed ledger from the public indexer in your browser): nightpass-demo.nightshift-labs.workers.dev/live — contract balance 500 NIGHT units = 2000 bought − 1500 withdrawn.

Demo



Live: nightpass-demo.nightshift-labs.workers.dev — the compiled contract running in the browser (local Midnight simulator), with three "try to cheat" buttons.

Market & adoption

- **Who pays:** API providers selling to agents (data, search, on-chain analytics) who want privacy-conscious agent customers.
- **Why now:** agent-to-API payments (x402) are growing; privacy is the missing piece.
- **Adoption path:** drop-in `X-Nightpass` header next to x402; we run our own paid agent API and will accept Nightpass first.
- **Model:** providers keep 100% of the bundle price; optional hosted verifier.

Roadmap & honest limits

- Next: Lace wallet in the UI so agents can redeem against the live preview contract from the page (read-only live view already shipped).
- Shielded payment inside the purchase, so buying is private too.
- Package the gate (verified against the preview indexer) as drop-in middleware next to x402.
- Bundle expiry, per-endpoint credit classes.
- Limits: purchase is public, unlinkability starts after it; anonymity set = bundles before the proven root; use a fresh fee wallet + hidden IP for redemptions.